香港城市大學
City University of Hong Kong

SDSC2102
Statistical Methods and Data Analysis

FINAL PROJECT

Housing Market Prices Analysis and Modeling

Course Instructor: Prof. ZENG Li

Group 9:
KOJONGIAN Gracelynn (57887115)
KURNIAWAN Erica Cleodora (57889286)
YOLANDA Valeryn (57884567)

Semester B (2024-2025)

1. Introduction

Hong Kong is one the most populated cities in the world, with a persistent housing affordability crisis driven by limited land supply and high demand. For over a decade, Hong Kong has ranked as the least affordable housing market in the world (Kwan 2021). The housing market in Hong Kong has long been characterized by soaring prices, making homeownership increasingly unattainable for the average citizen (Yiu, 2021).

Hong Kong's housing problems continue to persist despite the government's actions toward addressing them. For instance, the government has implemented various policies, such as stamp duties and subsidized housing schemes, to stabilize the market, yet housing affordability remains a critical issue (Hong Kong Free Press, 2024). Housing prices have shown an upward trend throughout, except during the Asian financial crisis that began in 1997. From their lowest point to their highest point, housing prices have increased sixfold (Wong, 2019).

Given the complexity of Hong Kong's housing market, this paper seeks to analyze historical housing price trends using transaction data from Centaline Property, one of the largest real estate agencies in Hong Kong. The dataset includes transaction dates, property locations, floor plans, saleable areas, unit rates, and district classifications, allowing for a comprehensive examination of price movements across different regions and property types. By examining key factors, the research aims to provide insights into the dynamics of Hong Kong's housing market prices.

2. Objective

This study aims to identify the key determinants influencing housing price disparities in Hong Kong by analyzing the predictive strength of contributing factors within its real estate market. In this research, the following questions will be addressed:

1. To what extent do historical transaction patterns serve as predictive indicators for housing price movements in Hong Kong's residential market?
2. Which specific transaction variables demonstrate the strongest correlation relationship with overall market price trends?
3. Can machine learning classification models accurately categorize Hong Kong housing units into price tiers (low, medium, high)?

3. Methodology

3.1 Dataset Explanation

This project analyzes Hong Kong's housing market using a dataset of 1,139 property listings, each described by 28 attributes. The variables include both categorical and continuous features. Categorical variables include property type (**`Is_Studio`**, **`Prop_type_estate`**, **`Prop_type_single`**), Roof status (**`Roof_N`**, **`Roof_Y`**), and proximity to amenities (**`Kindergarten`**). On the other hand, continuous variables consist of temporal metrics (**`Days_to_reg_date`**), physical attributes (**`Bedroom`**, **`SaleableArea`**, **`GrossArea`**, **`Floor`**, **`Build_Ages`**), pricing metrics (**`SaleableAreaPrice`**, **`GrossAreaPrice`**), locational features (**`Latitude`**, **`Longitude`**), nearby facilities (**`Primary_Schools`**, **`Wet_Market`**, **`Mall`**, **`Library`**, **`Parks`**, **`Bus_Route`**).

The study treats **`SaleableAreaPrice`** (price per unit of saleable area) as the dependent variable, as it reflects the market value relative to usable space. The remaining variables serve as predictors, testing how factors such as building characteristics, neighborhood amenities, and locational data influence housing prices.

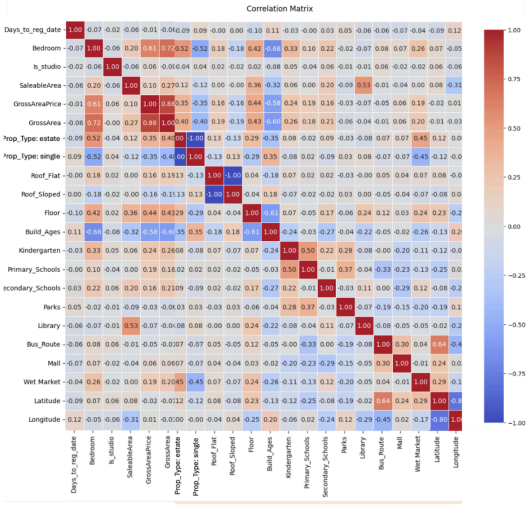
3.2 Data Preprocessing

For data preprocessing, categorical variables were encoded into numerical formats, with object-type columns systematically converted to integers or floats. Subsequently, to address the missing values, data imputation was implemented. First, entries were grouped by geographic location and the null values were imputed using location-specific averages to preserve regional pricing patterns. In this project, outliers were retained due to their critical role in reflecting Hong Kong's unique housing market volatility and premium pricing behaviors. Lastly, the target variable, which is housing prices, was categorized into three tiers; low, medium, high based on the quartile.

3.3 Data Visualization

3.3.1 Heat Map

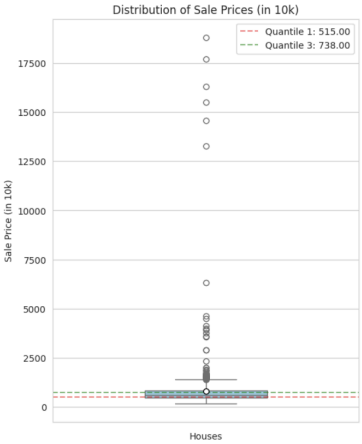
To visualize data and find hidden relationships, a correlation heatmap was constructed. A heatmap specifically used to quantify correlations between variables, revealing any hidden correlation. The heatmap highlights the strength and direction of linear relationships, ranging from -1 (perfect negative correlation) to +1 (perfect positive correlation). The analysis focused particularly on variables demonstrating correlation coefficients exceeding |0.5|, as these represent potentially meaningful relationships in the dataset.



The heatmap revealed strong relationships in Hong Kong's housing market. Notably, a strong positive correlation (r = 0.61) between gross area price and bedroom count, indicating each additional bedroom adds substantial value. This reflects the premium placed on living space in Hong Kong's space-constrained environment, with the nearly linear relationship showing consistent value increments per bedroom.

3.3.2 Box Plot

To complement our correlation analysis, we employed box plots to examine how SaleaPrice (Y-axis) varies across different numerical variables (X-axis). The box plot shows that the first quartile is 525 and the third quartile is 738. Those quartiles are used to determine the price tier where every price below the first quartile is considered as low, every price above the third quartile is considered as high, and every price in between is medium. In this analysis, outliers were retained due to their critical role in reflecting market volatility and premium pricing behaviors.



4. Data Modelling

Two machine learning methods were implemented: decision tree and linear regression. The dataset was divided into training (80%) and testing (20%) subsets to ensure a robust evaluation of the models.

4.1 Evaluation Metrics

The evaluation metrics for the training dataset are the root mean squared error (RMSE), mean absolute error (MAE), and R-squared (R2) metrics. For the test set, we calculate only RMSE and MAE. This ensures the model is unbiased on predicting unseen data. The metrics for the test set are given below:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \qquad MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

4.2 Linear Regression

Linear regression was used to predict housing prices based on various predictors. It also evaluated feature importance using the Ordinary Least Squares (OLS) methodology. The regression equation provided coefficients for each feature, allowing for the interpretation of their impact on housing prices as the following:

- x_4 **(SaleableArea)** displays the strongest positive impact on price (coefficient = 3.4424), indicating that each additional square foot of saleable area increases property value by approximately 3.44 HKD per unit, all else being equal.
- x_{12} **(Floor)** demonstrates a significant positive effect (4.3224), meaning higher-floor units command a premium of 4.32 HKD per unit per floor, reflecting the desirability of elevated views.
- x_{33} **(Build_Ages)** has a notable negative coefficient (-11.9578), implying that each additional year of building age reduces price by 11.96 HKD per unit, highlighting depreciation trends unless offset by heritage value or renovations.

To strengthen results, coefficients were standardized using Z-tests. Furthermore, from the top ten features ($p < 0.05$) that were deemed statistically significant, **SaleableArea** had the strongest impact, followed by **BuildingAge.**

$$\hat{Y} = 2.132e^{6} + 0.1035x1 + 3.0030x2 + 42.4754x3 + 3.4424x4 + 0.0340x5 + (-1.5095)x6 + (0.0045x7) + 1.066e^{6}x8 + 1.066e^{6}x9 + 1.066e^{6}x10 + 1.066e^{6}x11 + 4.3224x12 + (-11.9578)x13 + (-27.5615x14) + (-8.5643x15) + (-112.2028x16) + (-10.5116x17) + 690.4191x18 + (-0.0026)x19 + (-2.2347x20) + (-11.9815x21) + (-4.525e^{4}x22) + (-2.851e^{4}x23)$$

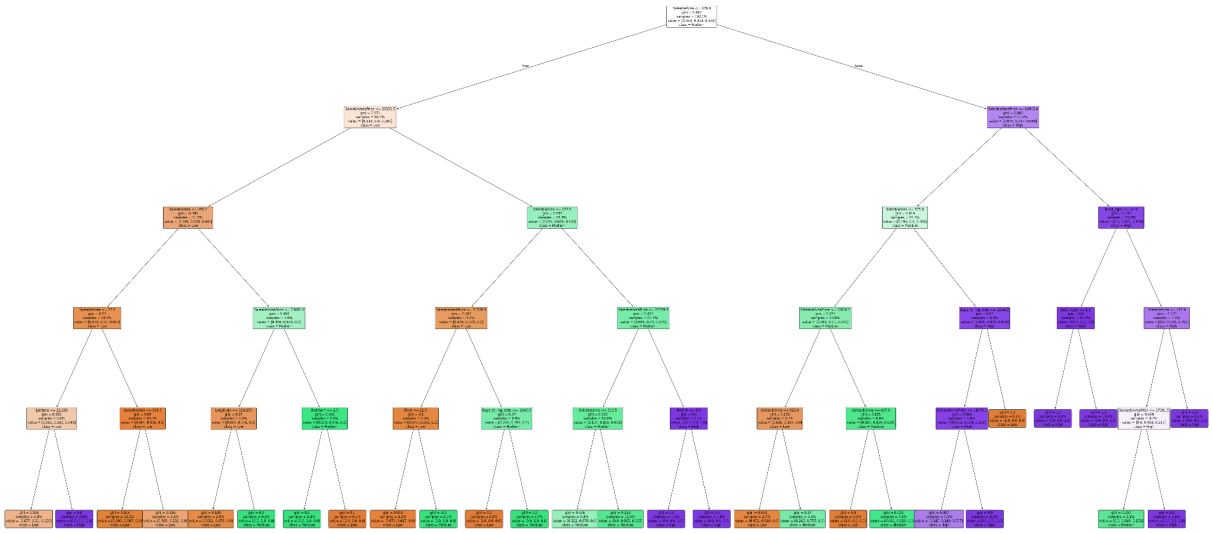
The model's performance is summarized in the following table:

Metric	Training Set	Test Set
RMSE	555.125	429.335
MAE	282.51	255.955
R^2	0.7636	0.9118

The model demonstrates moderate predictive accuracy on the test set, with an R^2 of 0.76 explaining approximately 76% of the variance in the data.

4.3 Decision Tree

The decision tree model was employed for categorical price classification. Prices were divided into three categories: low, medium, and high, based on quartile thresholds. This classification facilitates the understanding of price dynamics in a more interpretable format. The decision tree model demonstrated strong performance metrics with high precision across all price categories. Additionally, the model achieved an overall accuracy of 90%, indicating robust predictive capabilities.



The tree model illustrated that older buildings generally correspond to lower selling prices, while higher floors tended to ask for higher prices. Additionally, location, derived from longitude and latitude, played a significant role in determining property prices, as geographic factors often influence market demand and value. These insights provide a comprehensive understanding of market behavior and the factors impacting housing prices. But aside from that, the number of bedrooms in housing units was not really associated with higher or lower prices, highlighting bedroom does not have that much influence in the pricing decisions. This suggests that even though the number of bedroom seems to be an important factor of determining prices of unit that belongs in one building, but in the big housing market dynamics, there are other factors that are more significant than the number of bedroom

The model demonstrated high precision in predicting the three price classes (low/medium/high), where precision scores were 0.91, 0.83, and 0.97 respectively. This indicates that the model is highly reliable in predicting high instances, while its performance in the medium category is slightly lower. Furthermore, recall scores were 0.88 for low, 0.91 for medium, and 0.91 for high. Recall scores measure the model's ability to identify all relevant instances within each category. Hence, the scores suggest that the model is effective in capturing most instances, particularly in the medium and high categories, but with a slight drop in the low category. Lastly, the F1-score, which combines precision and recall into a single metric, offers a balanced view of the model's performance. The F-1 scores were 0.89 for low, 0.86 for medium, and 0.94 for high. The High category stands out with a notably high F1-score, indicating a strong balance between precision and recall.

Overall, the model achieves an accuracy of 0.90, signifying that 90% of predictions made by the model are correct. Both the macro and weighted averages for precision, recall, and F1-score also stand at 0.90, underscoring the model's consistent performance across all categories.

	precision	recall	f1-score	support
Low (0)	0.91	0.88	0.89	76
Medium (1)	0.83	0.91	0.86	74
High (2)	0.97	0.91	0.94	78
accuracy			0.90	228
macro avg	0.90	0.90	0.90	228
weighted avg	0.90	0.90	0.90	228

5. Conclusion

In conclusion, historical patterns serve as significant predictors for housing prices in Hong Kong, as it can be seen that both the machine learning models show quite high accuracy. Linear Regression model is used for determining the sales price while Decision Tree is used for determining the sale price tiers (low, medium, and high). From the Linear Regression result, it can be seen that the SaleableArea, BuildingAge, and GrossArea have the most significance in determining the SalePrice. Lastly, Decision Tree has a better accuracy compared to Linear Regression, indicating the relation between each variable follows a nonlinear relationship.

6. References

Hong Kong Free Press. (2024, January 15). Hong Kong's housing crisis: Prices dip but affordability remains worst in the world. https://hongkongfp.com

Yiu, C. Y. (2021). Housing market dynamics and government intervention in Hong Kong. Journal of Housing Economics, 52, 101738. https://doi.org/10.1016/j.jhe.2021.101738

Chan, S. M. (2022). The Long-Lasting housing problems in Hong Kong. In Springer eBooks (pp. 1–15). https://doi.org/10.1007/978-3-030-68127-2_245-1

Wong, M. B. (2019). What caused Hong Kong's housing crisis? Green Paper 2022 Hong Kong Economic Policy, 94–97. https://mbwong.com/pdf/Housing_Crisis.pdf

Appendix

Linear Regression Coefficient Details:

1. x_1 (Days_to_reg_date): The number of days from the property listing date to the registration date, indicating how long the property has been on the market.
2. x_2 (Bedroom): The number of bedrooms in the property, which is a key feature affecting housing prices.
3. x_3 (Is_studio): A binary indicator (0 or 1) indicating whether the property is a studio apartment. A value of 1 means it is a studio, while 0 means it is not.
4. x_4 (SaleableArea): The area of the property that can be sold, usually measured in square feet or square meters. This is a crucial factor influencing price.
5. x_5 (GrossArea): The total area of the property, including all spaces such as walls and common areas. It provides a broader view of the property size.
6. x_6 (SaleableAreaPrice): The price per unit of the saleable area, calculated to understand the market value relative to the usable space.
7. x_7 (GrossAreaPrice): The price per unit of the gross area, giving insights into pricing based on total property size.
8. x_8 (Prop_Type: estate): A categorical variable indicating whether the property is classified as an estate. This can affect the desirability and pricing of the property.
9. x_9 (Prop_Type: single): A categorical variable indicating whether the property is a single-family home, which can have different pricing dynamics compared to multi-family or commercial properties.
10. x_{10} (Roof: N): A binary indicator for properties without a roof type defined as "N." This classification may influence maintenance and pricing.
11. x_{11} (Roof: Y): A binary indicator for properties with a roof type defined as "Y." Similar to x_{10}, this classification can affect property value.
12. x_{12} (Floor): The floor number on which the property is located. Higher floors may command higher prices due to better views or reduced noise.
13. x_{13} (Build_Ages): The age of the building, which can impact its value. Older buildings may have different pricing dynamics compared to newer constructions.
14. x_{14} (Kindergarten): A binary indicator (0 or 1) indicating proximity to kindergartens, which can influence family housing decisions and property values.
15. x_{15} (Primary_Schools): The proximity to primary schools, a significant factor for families looking to buy homes in certain neighborhoods.
16. x_{16} (Secondary_Schools): The proximity to secondary schools, which can also affect family decisions regarding property purchases.
17. x_{17} (Parks): The proximity to parks, which can enhance the desirability of a neighborhood and affect housing prices.

18. x_{18} (Library): The distance to the nearest library, a factor that may be considered by families or individuals valuing education and community resources.
19. x_{19} (Bus_Route): Proximity to bus routes, which can greatly influence convenience and accessibility for residents, impacting property value.
20. x_{20} (Mall): The distance to the nearest shopping mall, which can enhance the attractiveness of a location for potential buyers.
21. x_{21} (Wet_Market): Proximity to wet markets, which are often valued in certain cultures for convenience and availability of fresh produce.
22. x_{22} (Latitude): The geographic latitude of the property, used in spatial analysis and to determine location-based pricing dynamics.
23. x_{23} (Longitude): The geographic longitude of the property, similarly important for spatial analysis and understanding market trends based on location.

Part 1: Data Cleaning

```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import datetime
```

```
property_hk = pd.read_csv("HKProp_Datasetuse.csv")
property_hk.dtypes
```

	0
Reg_Date	object
Reg_Year	int64
Prop_Name_ENG	object
ADDRESS_ENG	object
Prop_Type	object
Estate_Size	int64
Tower	object
Floor	int64
Flat	object
Bed_Room	float64
Roof	object
Build_Ages	int64
Rehab_Year	float64
SalePrice_10k	int64
SaleableArea	object
Gross Area	object
SaleableAreaPrice	object
Gross Area_Price	object
Kindergarten	int64
Primary_Schools	int64

Primary_Schools	int64
Secondary_Schools	int64
Parks	int64
Library	int64
Bus_Route	int64
Mall	int64
Wet Market	int64
Latitude	float64
Longitude	float64

dtype: object

```
property_hk_cleaned = property_hk.replace('--', '0').replace('$', '')


Reg_Date = pd.to_datetime(np.array(property_hk_cleaned['Reg_Date']))
Days_to_reg_date = [int(t.days) for t in (Reg_Date - datetime.datetime.today())
```

```
<ipython-input-4-ed07e4bc6474>:1: UserWarning: Parsing dates in %d/%m/%Y fo
  Reg_Date = pd.to_datetime(np.array(property_hk_cleaned['Reg_Date']))
```

```
Bedroom = property_hk_cleaned['Bed_Room'].replace('Studio', '0').fillna(0)
Is_studio = [1 if e == 0 else 0 for e in np.array(property_hk_cleaned['Bed_Room
```

```
SaleableArea = [int(str(t).replace(',', '').replace('nan', '0')) for t in prope
SaleableAreaPrice = [int(str(t).replace(',', '').replace('$', '0').replace('nan
GrossArea = [int(str(t).replace(',', '').replace('nan', '0')) for t in property
GrossAreaPrice = [int(str(t).replace(',', '').replace('$', '0').replace('nan',

# Convert SaleableAreaPrice to float
property_hk_cleaned['SaleableAreaPrice'] = (
    property_hk_cleaned['SaleableAreaPrice']
    .astype(str)
    .str.replace(',', '')
    .str.replace('$', '')
    .replace('nan', '0')
    .astype(float)
)

# Convert GrossAreaPrice to float (same process)
property_hk_cleaned['GrossAreaPrice'] = (
    property_hk_cleaned['Gross Area_Price']
    .astype(str)
    .str.replace(',', '')
    .str.replace('$', '')
    .replace('nan', '0')
    .astype(float)
)

# Verify conversion
print(property_hk_cleaned[['SaleableAreaPrice', 'GrossAreaPrice']].head())
print(f"\nData types:\n{property_hk_cleaned[['SaleableAreaPrice', 'GrossAreaPri
```

```
   SaleableAreaPrice  GrossAreaPrice
0            14763.0             0.0
1            16156.0         10307.0
2            17868.0         11398.0
3                0.0             0.0
4            19459.0             0.0

Data types:
SaleableAreaPrice    float64
GrossAreaPrice       float64
dtype: object
```

```
property_hk_cleaned['Prop_Type'] = property_hk_cleaned['Prop_Type'].str.lower()
```

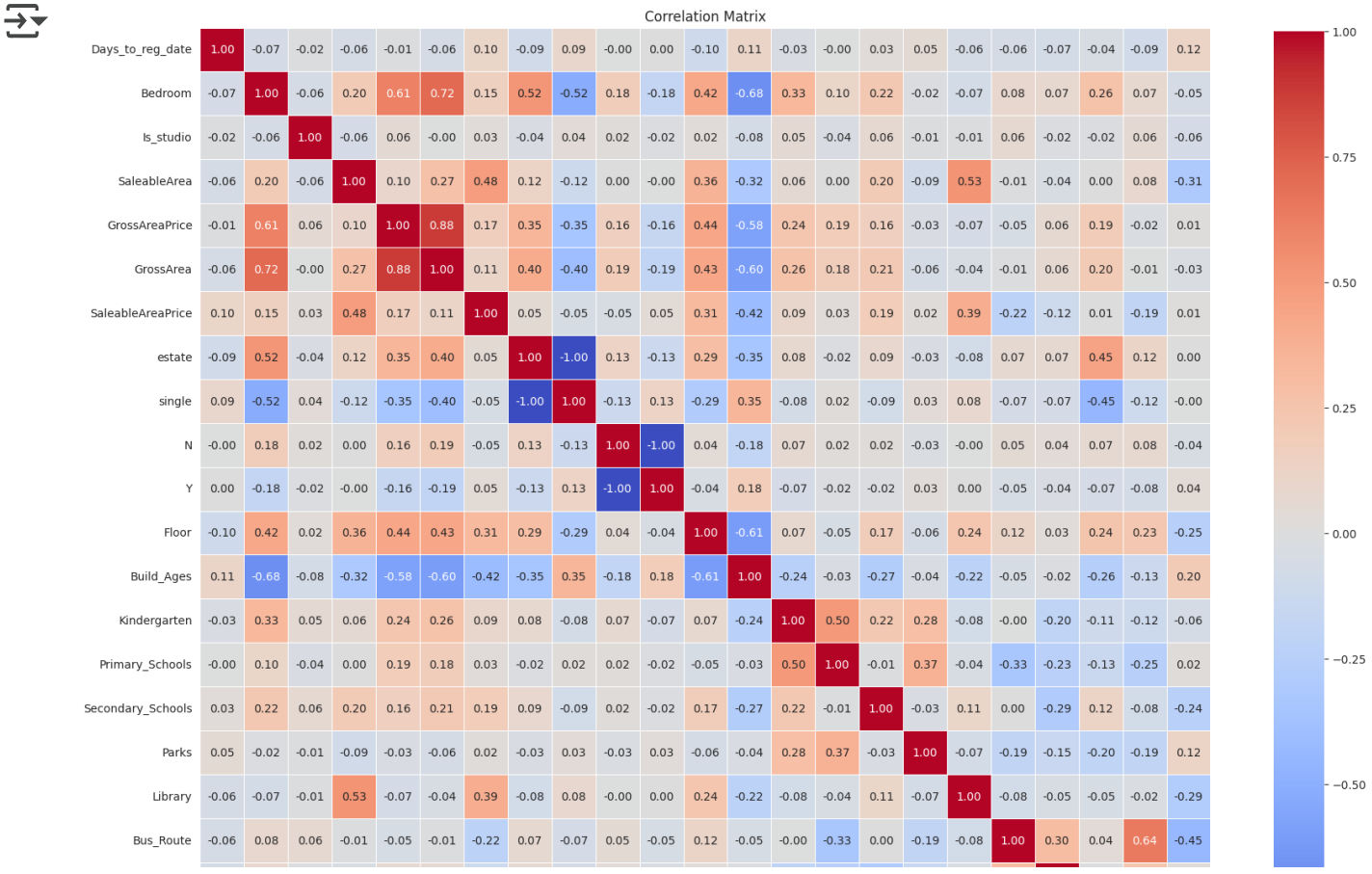
property_hk_cleaned

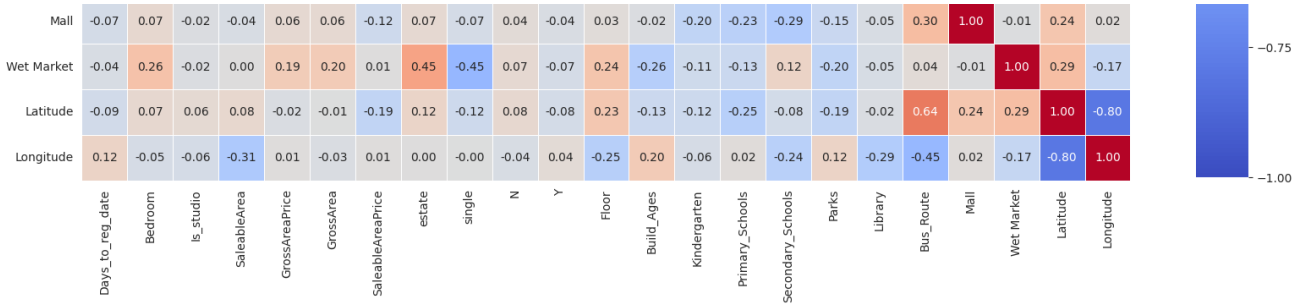
	Reg_Date	Reg_Year	Prop_Name_ENG	ADDRESS_ENG	Prop_Type	Estate_Size
0	26/10/2016	2016	18 SHELLEY STREET	18 SHELLEY STREET	single	1
1	16/11/2017	2017	Lilian Court	6-8 SHELLEY STREET	single	1
2	11/10/2016	2016	Lilian Court	6-8 SHELLEY STREET	single	1
3	18/10/2017	2017	Felicity Building	9-13 SHELLEY STREET	single	1
4	18/10/2017	2017	9-13 SHELLEY STREET	9-13 SHELLEY STREET	single	1
...	...	...	...	...	...	...
1134	8/8/2017	2017	Caine Tower	55 ABERDEEN STREET	single	1
1135	28/4/2017	2017	Caine Tower	55 ABERDEEN STREET	single	1
1136	22/3/2017	2017	Caine Tower	55 ABERDEEN STREET	single	1
1137	15/2/2017	2017	Caine Tower	55 ABERDEEN STREET	single	1
1138	5/6/2016	2016	Caine Tower	55 ABERDEEN STREET	single	1

1139 rows × 29 columns

Part 2: Visualization

```
X = np.concatenate([np.array(Days_to_reg_date).reshape(-1, 1), np.array(Bedroom
                    np.array(Is_studio).reshape(-1, 1), np.array(SaleableAr
                    np.array(GrossAreaPrice).reshape(-1, 1),
                    np.array(GrossArea).reshape(-1, 1),
                    np.array(SaleableAreaPrice).reshape(-1, 1),
                    np.array(pd.get_dummies(property_hk_cleaned[['Prop_Type
                    np.array(property_hk_cleaned[['Floor', 'Build_Ages', 'K
                                                 'Parks', 'Library', 'Bus_


columns = ['Days_to_reg_date', 'Bedroom', 'Is_studio', 'SaleableArea', 'GrossAr
           'GrossArea', 'SaleableAreaPrice', 'estate', 'single', 'N', 'Y',
           'Floor', 'Build_Ages', 'Kindergarten', 'Primary_Schools',
           'Secondary_Schools', 'Parks', 'Library', 'Bus_Route',
           'Mall', 'Wet Market', 'Latitude', 'Longitude']

df = pd.DataFrame(X, columns=columns)

correlation_matrix = df.corr()

plt.figure(figsize=(20, 20))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=

plt.title('Correlation Matrix')
plt.show()
```





Part 3: Machine Learning

Linear Regression

```
from sklearn.model_selection import train_test_split
```

```
import statsmodels.api as sm

# Define Target Variable (y)
y = property_hk_cleaned['SalePrice_10k'].values

# Split Data into Train/Test Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# OLS Regression
X_sm = sm.add_constant(X_train)
ols_model = sm.OLS(y_train, X_sm).fit()
print(ols_model.summary())
```

OLS Regression Results

Dep. Variable:	y	R-squared:	0.
Model:	OLS	Adj. R-squared:	0.
Method:	Least Squares	F-statistic:	13
Date:	Fri, 28 Mar 2025	Prob (F-statistic):	4.42e-
Time:	07:28:19	Log-Likelihood:	-704
No. Observations:	911	AIC:	1.414e
Df Residuals:	889	BIC:	1.425e
Df Model:	21		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.9
const	2.132e+06	1.21e+06	1.769	0.077	-2.34e+05	4.5e
x1	0.1035	0.097	1.067	0.286	-0.087	0.
x2	3.0030	32.403	0.093	0.926	-60.592	66.
x3	42.4754	223.024	0.190	0.849	-395.240	480.
x4	3.4424	0.117	29.369	0.000	3.212	3.
x5	0.0340	0.008	4.443	0.000	0.019	0.
x6	-1.5095	0.168	-8.968	0.000	-1.840	-1.
x7	0.0045	0.004	1.186	0.236	-0.003	0.
x8	1.066e+06	6.03e+05	1.769	0.077	-1.17e+05	2.25e
x9	1.066e+06	6.03e+05	1.769	0.077	-1.17e+05	2.25e
x10	1.066e+06	6.03e+05	1.769	0.077	-1.17e+05	2.25e
x11	1.066e+06	6.03e+05	1.769	0.077	-1.17e+05	2.25e
x12	4.3224	2.579	1.676	0.094	-0.739	9.
x13	-11.9578	2.324	-5.146	0.000	-16.518	-7.
x14	-27.5615	15.159	-1.818	0.069	-57.314	2.
x15	-8.5643	15.718	-0.545	0.586	-39.414	22.
x16	-112.2028	49.254	-2.278	0.023	-208.870	-15.
x17	10.5116	4.408	2.385	0.017	1.861	19.
x18	690.4191	100.791	6.850	0.000	492.603	888.
x19	-0.0026	0.937	-0.003	0.998	-1.842	1.
x20	-2.2347	88.788	-0.025	0.980	-176.493	172.
x21	-11.9815	90.052	-0.133	0.894	-188.721	164.
x22	-4.525e+04	2.83e+04	-1.598	0.110	-1.01e+05	1.03e
x23	-2.851e+04	1.61e+04	-1.774	0.076	-6.01e+04	3030.

Omnibus:	825.431	Durbin-Watson:	1.
Prob(Omnibus):	0.000	Jarque-Bera (JB):	49637.
Skew:	3.856	Prob(JB):	0
Kurtosis:	38.330	Cond. No.	1.59e

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corr
[2] The smallest eigenvalue is 1.31e-29. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
y_train_pred = ols_model.predict(X_sm)
X_test_sm = sm.add_constant(X_test)
y_test_pred = ols_model.predict(X_test_sm)

train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))  # Alternative
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))
train_mae = mean_absolute_error(y_train, y_train_pred)
test_mae = mean_absolute_error(y_test, y_test_pred)
train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)  # Now including test R²

# Create Table 1
performance_table = pd.DataFrame({
    'Metric': ['RMSE', 'MAE', 'R²'],
    'Training': [f"{train_rmse:.4f}", f"{train_mae:.4f}", f"{train_r2:.4f}"],
    'Test': [f"{test_rmse:.4f}", f"{test_mae:.4f}", f"{test_r2:.4f}"]
})

# Display the table in markdown format
print("Model Performance")
print("\nTable 1: OLS Regression Performance Metrics\n")
print(performance_table.to_markdown(index=False))
```

```
Model Performance

Table 1: OLS Regression Performance Metrics

| Metric   |   Training |     Test |
|:---------|-----------:|---------:|
| RMSE     |   555.125  | 429.335  |
| MAE      |   282.51   | 255.955  |
| R²       |     0.7636 |   0.9118 |
```

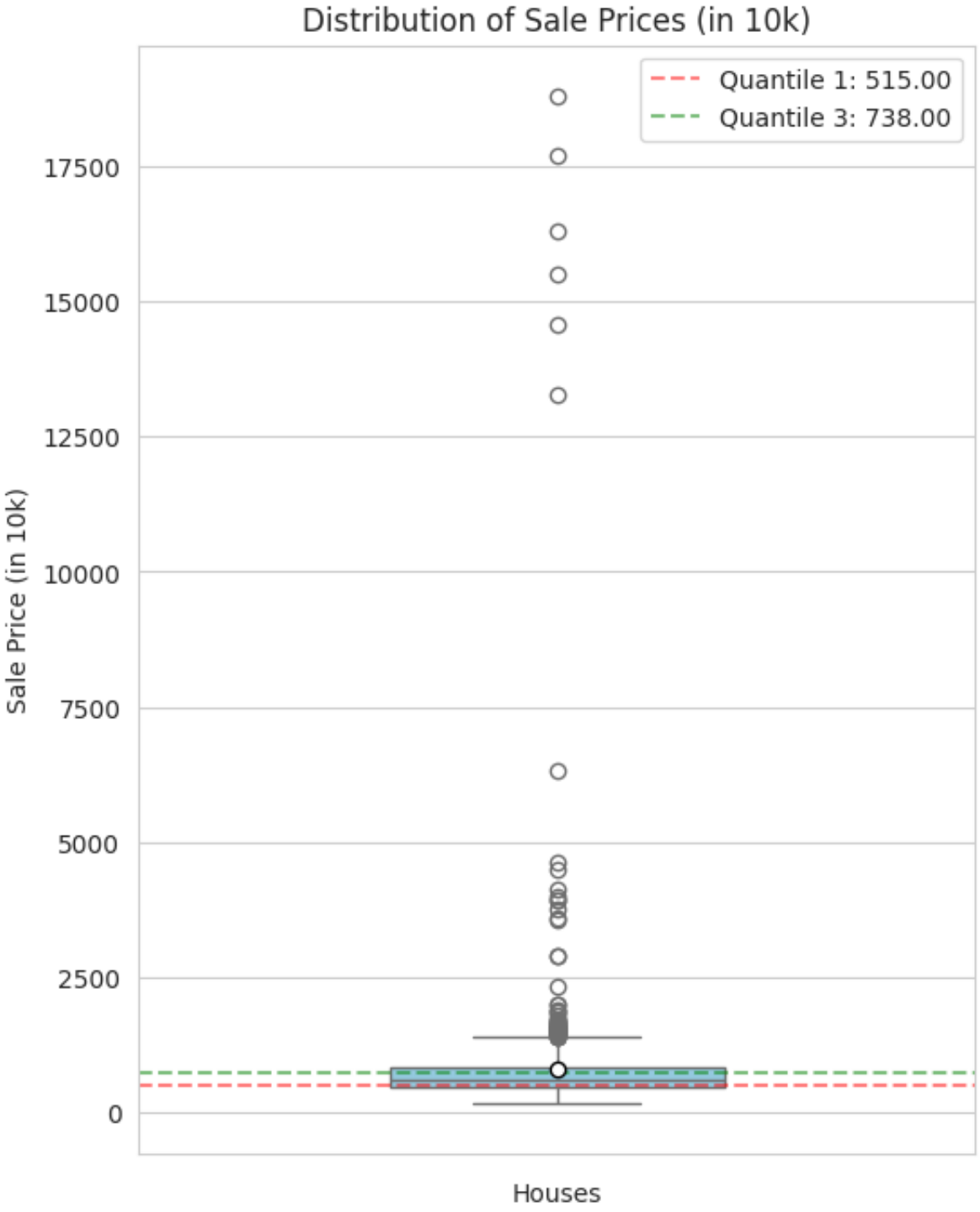
```
# Feature importance
feature_names = ['Days_to_reg_date', 'Bedroom', 'Is_studio', 'SaleableArea', 'G
feature_names = ['const'] + feature_names

importance_df = pd.DataFrame({
    'Feature': feature_names,
    'P_value': ols_model.pvalues
})

top_10_pvalue = importance_df[importance_df['Feature'] != 'const'].sort_values(
    'P_value', ascending=True
).head(10)

print("\nTop 10 Statistically Significant Features (p < 0.05):")
print(top_10_pvalue[['Feature', 'P_value']])
```

```
Top 10 Statistically Significant Features (p < 0.05):
              Feature        P_value
4        SaleableArea  4.666674e-133
6           GrossArea   1.753124e-18
18            Library   1.377935e-11
13          Build_Ages   3.272159e-07
5      GrossAreaPrice   9.992028e-06
17              Parks   1.729786e-02
16  Secondary_Schools   2.295983e-02
14       Kindergarten   6.938270e-02
23          Longitude   7.639265e-02
11             Roof_Y   7.723220e-02
```

** Decision Tree**

```
df = property_hk_cleaned.copy()

low_thresh = df['SalePrice_10k'].quantile(0.33)
high_thresh = df['SalePrice_10k'].quantile(0.66)

plt.figure(figsize=(6, 8))
sns.set_style("whitegrid")
box_plot = sns.boxplot(y='SalePrice_10k', data=df, color='skyblue', width=0.4,
plt.title('Distribution of Sale Prices (in 10k)')
plt.ylabel('Sale Price (in 10k)')
plt.xlabel('Houses')
plt.axhline(low_thresh, color='red', linestyle='--', alpha=0.5, label=f'Quantil
plt.axhline(high_thresh, color='green', linestyle='--', alpha=0.5, label=f'Quar
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7c33abb42390>
```



```
df['Price_Tier'] = np.select(
    [
        df['SalePrice_10k'] <= low_thresh,
        df['SalePrice_10k'] >= high_thresh
    ],
    [0, 2],
    default=1
).astype(int)

print("Low Price (0): below ", low_thresh, " (in 10k)")
print("Medium Price (1): from ", low_thresh, " to ", high_thresh, " (in 10k)")
print("High Price (2): above ", high_thresh, " (in 10k)")

print("\nDistribution:")
print(df['Price_Tier'].value_counts().sort_index())
```

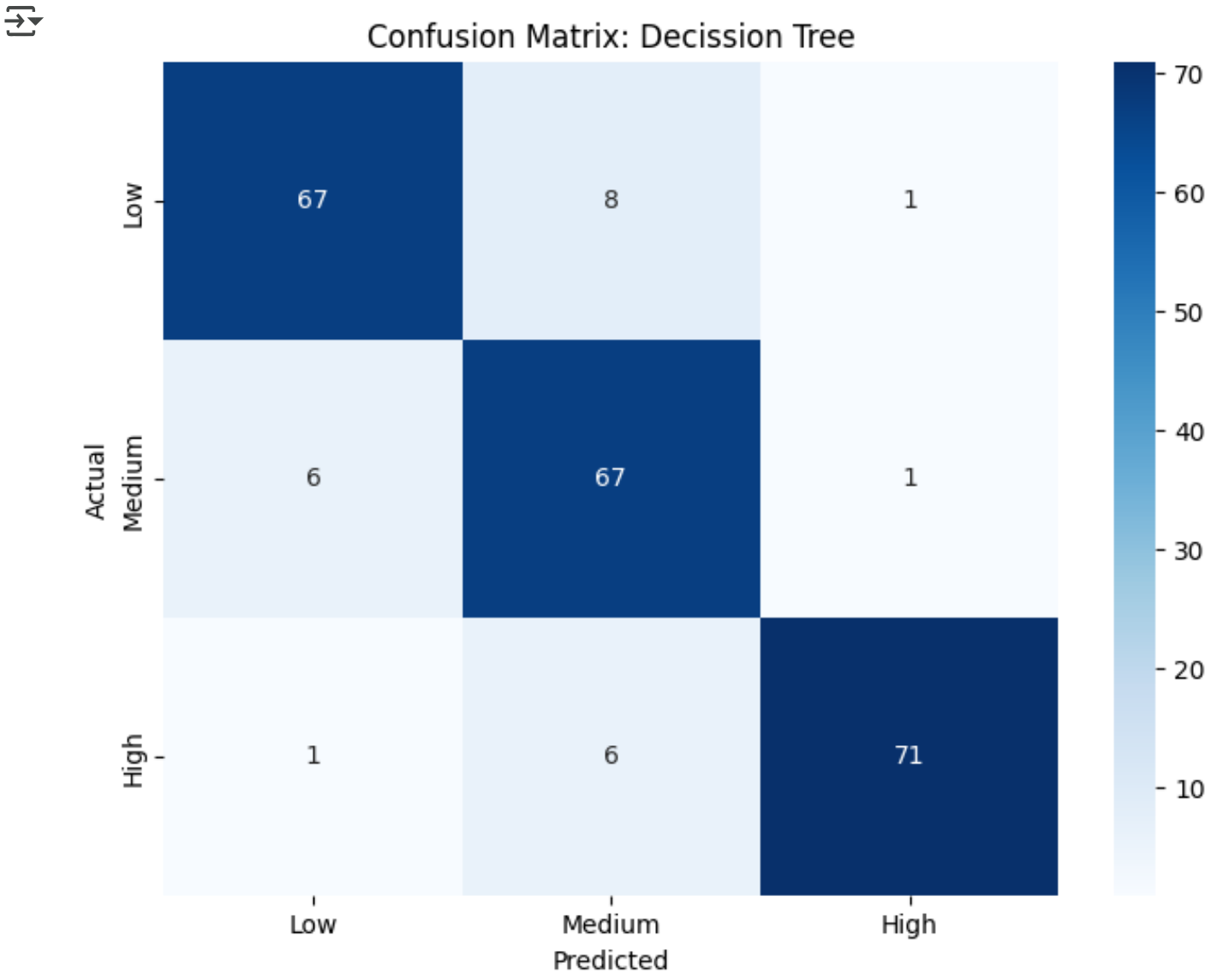
```
Low Price (0): below  515.0  (in 10k)
Medium Price (1): from  515.0  to  738.0  (in 10k)
High Price (2): above  738.0  (in 10k)

Distribution:
Price_Tier
0    380
1    370
2    389
Name: count, dtype: int64
```

```
X1 = np.concatenate([np.array(Days_to_reg_date).reshape(-1, 1), np.array(Bedroc
                     np.array(Is_studio).reshape(-1, 1), np.array(SaleableAr
                     np.array(GrossAreaPrice).reshape(-1, 1),
                     np.array(GrossArea).reshape(-1, 1),
                     np.array(SaleableAreaPrice).reshape(-1, 1),
                     np.array(pd.get_dummies(property_hk_cleaned[['Prop_Type
                     np.array(property_hk_cleaned[['Floor', 'Build_Ages', 'K
                                                  'Parks', 'Library', 'Bus_R

y1 = df['Price_Tier'].values

# Train-test split (stratified to preserve tier ratios)
X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X1, y1,
    test_size=0.2,
    random_state=42,
    stratify=y1
)
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

model = DecisionTreeClassifier(
    max_depth=5,
    class_weight='balanced',
    random_state=42
)
model.fit(X_train1, y_train1)

# Evaluate
y_pred1 = model.predict(X_test1)
print("Model Performance:")
print(classification_report(
    y_test1,
    y_pred1,
    target_names=['Low (0)', 'Medium (1)', 'High (2)']
))
```

```
Model Performance:
                precision    recall  f1-score   support

     Low (0)       0.91      0.88      0.89        76
  Medium (1)       0.83      0.91      0.86        74
    High (2)       0.97      0.91      0.94        78

    accuracy                           0.90       228
   macro avg       0.90      0.90      0.90       228
weighted avg       0.90      0.90      0.90       228
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test1, y_pred1)
plt.figure(figsize = (8, 6))
sns.heatmap(cm, annot = True, fmt = 'd', cmap = 'Blues', xticklabels = ['Low',
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix: Decission Tree')
plt.show()
```




```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(80, 40))
plot_tree(
    model,
    feature_names = ['Days_to_reg_date', 'Bedroom', 'Is_studio', 'SaleableArea'
```

```
    + list(pd.get_dummies(property_hk_cleaned[['Prop_Type', 'Roof']]).columns)
    + ['Floor', 'Build_Ages', 'Kindergarten', 'Primary_Schools', 'Secondary_Sch
    class_names = ['Low', 'Medium', 'High'],
    filled=True,
    rounded=True,
    fontsize=12,
    proportion=True
)
plt.title("Decision Tree for Price Tiers (0=Low, 1=Medium, 2=High)", fontsize=1
plt.show()
```